

PATNA METRO MANAGEMENT SYSTEM

Project Working Document
Database Design & Management | Full-Stack Implementation

Project	Patna Metro Management System
Domain	Database Design & Management (DBMS)
Tech Stack	MySQL 8.0 FastAPI (Python) React.js
Database	13 BCNF Tables 4 Stored Procedures 3 Views Targeted Indexes
Interfaces	Admin Dashboard Vending Machine Kiosk (x2) Travel Card App
Version	v2.0 2026

1. Project Objectives

The Patna Metro Management System is a full-stack database-driven application built as a Database Design and Management project. The core objectives are:

1. Design a BCNF-normalized relational schema of 13 tables with enforced functional dependencies, eliminating all partial and transitive dependencies across entities such as Fare_Matrix, Card_Trips, and Audit_Log.
2. Implement ACID-compliant stored procedures (sp_card_tap_in, sp_card_tap_out, sp_recharge_card, sp_issue_ticket) using START TRANSACTION, COMMIT, ROLLBACK, and CHECK constraints to guarantee data consistency on every write operation.
3. Demonstrate InnoDB row-level locking via SELECT ... FOR UPDATE with fixed lock ordering (Card_Trips then Metro_Cards always) to serialize concurrent transactions and prevent lost-update anomalies without table-level locks.
4. Model a self-auditing financial ledger in Audit_Log with signed amount_delta, balance_after snapshots, and entity-level references ensuring every debit and credit is fully traceable and verifiable.
5. Expose indexed relational data through a REST API backed by composite indexes on Card_Trips(card_id, status) and Audit_Log(created_at), enabling sub-millisecond query response for live dashboard polling at 3 to 5 second intervals across four concurrent client interfaces.

2. Technology Stack

Layer	Technology	Role
Database	MySQL 8.0 InnoDB	Primary data store — ACID transactions, stored procedures, views, indexes
Backend	FastAPI (Python 3.11)	REST API — 40+ endpoints, connection pooling, pydantic validation
Frontend	React.js (CRA)	3 role-specific SPAs — live polling, no localStorage
ORM Layer	Raw SQL + mysql-connector	Direct queries and stored procedure calls — no ORM abstraction
Protocol	JSON over HTTP	Shared api.js client — single BASE_URL change for multi-laptop WiFi
Charts	Recharts	Area, Bar, Pie charts for admin revenue analytics

3. Database Design

3.1 Normalization — BCNF

Every table in the schema satisfies Boyce-Codd Normal Form: for every non-trivial functional dependency $X \rightarrow Y$, X is a superkey. This eliminates all redundancy and ensures every fact is stored exactly once.

Key Decomposition Decisions

- Fare_Matrix uses composite key (from_station_id, to_station_id) with canonical form enforced by CHECK (from_station_id < to_station_id). A fare is a property of a pair of stations, not of any single station.
- Card_Recharge_Log is separate from Metro_Cards because one card has many recharges — a repeating fact. It also carries a self-auditing constraint: CHECK (balance_after = balance_before + amount), making corrupt recharge records impossible to insert.
- Users and Admins are separate tables. Different functional dependencies, different authentication attributes, and no shared nullable columns — merging them would violate BCNF via transitive dependency.
- Audit_Log is a polymorphic append-only ledger. Every financial event (tap-in, fare deduction, recharge, ticket sale) writes one row with entity_type, entity_id, event_type, amount_delta, and balance_after.
- Liness_Stations resolves the many-to-many between Liness and Stations as a proper junction table.

3.2 Schema — 13 Tables

Table	Type	Purpose
Liness	Master	Metro lines — name, colour, active status
Liness_Stations	Junction	Many-to-many: lines and stations with stop sequence
Stations	Master	Station name, coordinates
Interchange_Stations	Master	Stations where passengers can switch lines
Fare_Matrix	Lookup	BCNF fare table: composite PK (from < to), single fare per pair
Users	Master	Passenger details — name, email, phone, 4-digit PIN
Metro_Cards	Transactional	Smart card linked to user — balance, status (ACTIVE/BLOCKED/EXPIRED)
Card_Recharge_Log	Audit	Every recharge with balance snapshot — self-auditing CHECK constraint
Card_Trips	Transactional	Tap-in/out records — entry/exit station, time, fare, status

Vending_Machines	Master	Kiosk registry — station, code, status (ONLINE/OFFLINE/MAINTENANCE)
Tickets	Transactional	Single-journey tickets — ticket_code (UUID), fare, valid_until
Admins	Master	Admin accounts — role, password hash
Audit_Log	Ledger	Immutable financial ledger — signed amount_delta, balance_after

3.3 Indexes

Index	Why it exists
Card_Trips(card_id, status)	Every tap-in checks for an IN_PROGRESS trip on this card. Without this composite index, MySQL full-scans Card_Trips on every tap-in.
Audit_Log(created_at)	Admin date-range filters on the audit log. Without this, filtering millions of rows is a full table scan.
Metro_Cards(user_id)	Card lookups by user during login and balance polling.
Tickets(machine_id, issued_at)	Machine-filtered ticket history on the vending kiosk.

4. ACID Properties

4.1 Atomicity

Every stored procedure wraps all writes in a single transaction. `sp_card_tap_out` performs three writes: deduct balance in `Metro_Cards`, update status in `Card_Trips`, insert into `Audit_Log`. If the server crashes after the first write, InnoDB replays the redo log on restart and rolls everything back. The passenger is never charged without a matching trip record.

4.2 Consistency

Three enforcement layers prevent invalid states from ever reaching the disk:

- Schema constraints: CHECK (`balance >= 0`), CHECK (`fare_amount >= 0`), CHECK (`from_station_id < to_station_id`), CHECK (`pin REGEXP '[0-9]{4}$'`), CHECK (`balance_after = balance_before + amount`).
- Foreign keys with ON DELETE RESTRICT: cannot delete a user who owns cards, cannot delete a card with active trips, cannot delete a station with trip history.
- Business logic inside procedures: card must be ACTIVE, balance $\geq$ Rs.10 before tap-in, no IN_PROGRESS trip already open, machine must be ONLINE to issue a ticket.

4.3 Isolation

InnoDB default isolation level is Repeatable Read. All stored procedures escalate to full row-level exclusion using `SELECT ... FOR UPDATE` before reading any balance or status.

Without locks, two simultaneous tap-ins on the same card both read balance = Rs.50, both pass the Rs.10 check, and both insert trips — creating two open trips. With `SELECT ... FOR UPDATE`, Session B is blocked until Session A commits. Session B then reads the card as IN_PROGRESS and rolls back.

Deadlock prevention: all procedures acquire locks in the same fixed order — `Card_Trips` first, then `Metro_Cards`. When all transactions follow the same acquisition order, circular waits are impossible.

4.4 Durability

InnoDB uses Write-Ahead Logging. Before acknowledging a COMMIT, MySQL flushes the change to the redo log on disk. A power cut one millisecond after a successful tap-out leaves the fare deduction intact when the server restarts — the redo log is replayed automatically.

4.5 OLTP vs OLAP

OLTP	OLAP
------	------

What	Tap-in, tap-out, recharge, ticket issue	Revenue summaries, daily trends, station traffic
Query	Short row-level reads/writes via stored procedures	Aggregations: SUM, COUNT, GROUP BY across date ranges
Tables	Metro_Cards, Card_Trips, Tickets	vw_daily_revenue, vw_station_traffic, vw_card_balances
Isolation	Row-level locks, transactions, ROLLBACK	Read-only, no locks needed, served from views

5. Stored Procedures

The application never writes directly to `Metro_Cards` or `Card_Trips`. All financial operations go through these four procedures. The backend calls them via dedicated `autocommit=False` connections using session variables (`SET @var / CALL / SELECT @var`) to guarantee OUT parameter visibility across all MySQL 8.x connector versions.

sp_card_tap_in (IN: card_id, station_id | OUT: trip_id, result_code, result_msg)

- Lock `Metro_Cards` row FOR UPDATE
- Validate: card EXISTS, status = ACTIVE, balance >= Rs.10
- Check `Card_Trips`: no IN_PROGRESS trip exists for this card
- INSERT into `Card_Trips` (status = IN_PROGRESS)
- INSERT into `Audit_Log` (TAP_IN event)
- COMMIT, return trip_id

sp_card_tap_out (IN: trip_id, exit_station_id | OUT: fare, result_code, result_msg)

- Lock `Card_Trips` row FOR UPDATE — read card_id and entry_station
- Validate: trip EXISTS, status = IN_PROGRESS, exit station != entry station
- Lookup fare from `Fare_Matrix` using LEAST/GREATEST for canonical pair
- Lock `Metro_Cards` FOR UPDATE (consistent order: Trips first, Cards second)
- If balance < fare, charge max(balance, Rs.5) — minimum recovery policy
- UPDATE `Metro_Cards`: balance -= fare
- UPDATE `Card_Trips`: exit_station, exit_time, fare_deducted, status = COMPLETED
- INSERT into `Audit_Log` (FARE_DEDUCT event with balance_after snapshot)
- COMMIT, return fare charged

sp_recharge_card (IN: card_id, amount, remarks | OUT: result_code, result_msg)

- Validate amount > 0
- Lock `Metro_Cards` FOR UPDATE
- UPDATE `Metro_Cards`: balance += amount
- INSERT into `Card_Recharge_Log` (balance_before, balance_after, amount — CHECK constraint enforces arithmetic)
- INSERT into `Audit_Log` (RECHARGE event)
- COMMIT

sp_issue_ticket (IN: machine_id, from_station_id, to_station_id | OUT: ticket_code, fare, result_code, result_msg)

- Generate ticket_code = UUID() BEFORE START TRANSACTION — no lock needed for pure computation
- Lock `Vending_Machines` FOR UPDATE — verify status = ONLINE
- Validate from != to
- Lookup fare from `Fare_Matrix`, default Rs.20 if pair not in matrix

- INSERT into Tickets (valid_until = NOW() + 4 hours)
- INSERT into Audit_Log (TICKET_SALE event)
- COMMIT, return ticket_code and fare

6. Data Flow — Database to Frontend

6.1 Request Pipeline

- 6. React component calls a method on the shared api.js client
- 7. api.js sends HTTP request to FastAPI backend at BASE_URL:8000
- 8. FastAPI router validates the request body using Pydantic schema
- 9. Router calls utils/database.py — either get_all, get_one, execute_write, or call_proc
- 10. MySQL executes the query or stored procedure against the InnoDB engine
- 11. Result rows are returned as Python dicts and serialised to JSON by FastAPI
- 12. api.js parses the JSON response and returns it to the React component
- 13. Component calls setState — React re-renders with new data

6.2 Live Polling Strategy

All three frontends use useRef-based setInterval polling to avoid memory leaks. Background polls are silent — they never set loading=true — so existing data stays visible while the next update loads.

Interface	Interval	What refreshes
Admin — Revenue / Trips / Tickets / Audit	5 seconds	All analytics, tables, KPI cards — silent background polls
Admin — Ticker bar	5 seconds	All-time totals in sidebar and scrolling marquee
Vending Machine	3 seconds	Machine status + recent tickets (paused during ticket display at step 3)
Travel Card	3 seconds	Card balance + active trip status + trip history

7. System Architecture

7.1 Component Overview

Laptop	Interface	Description
Laptop 1	Admin Dashboard	Revenue charts, trips/tickets tables, full CRUD on all entities, audit log with delete, live ticker
Laptop 2A	Vending Machine 1	3-step ticket wizard, fare lookup, ticket print screen, polls machine status every 3s
Laptop 2B	Vending Machine 2	Identical to 2A but URL param ?machine=2 to bind to a different machine record
Laptop 4	Travel Card App	PIN-based card login, tap-in/tap-out, recharge, trip history, live balance

All four interfaces connect to a single FastAPI backend running on one laptop over shared WiFi. To point a frontend at a different machine, change `BASE_URL` in `api.js` to the backend laptop's local IP address and ensure port 8000 is allowed through Windows Firewall.

7.2 Backend Module Structure

File / Module	Responsibility
<code>main.py</code>	FastAPI app entry point — registers all routers, loads <code>.env</code> , runs <code>init_db</code> on startup
<code>utils/database.py</code>	Connection pool (<code>autocommit=True</code> for reads) + dedicated <code>autocommit=False</code> connection for stored procedure calls to guarantee session variable visibility
<code>models/schemas.py</code>	All Pydantic v1 request/response models — <code>Ticket.id</code> is Optional to handle timing-miss fallback
<code>routers/stations.py</code>	GET / POST / PUT / DELETE /stations — with trip-history guard on delete
<code>routers/fares.py</code>	Fare matrix CRUD + <code>/fares/lookup</code> endpoint — always normalises to LEAST/GREATEST pair
<code>routers/cards.py</code>	Card CRUD + <code>/cards/{id}/recharge</code> — recharge routes through <code>sp_recharge_card</code> proc
<code>routers/trips.py</code>	<code>/trips/tap-in</code> and <code>/trips/tap-out</code> call stored procedures; GET <code>/trips</code> with date/card/status filters
<code>routers/tickets.py</code>	POST <code>/tickets</code> calls <code>sp_issue_ticket</code> with two-layer fallback: proc result then DB fetch then station-name lookup

routers/analytics.py	revenue-summary, daily-revenue (groups by entry_time for in-progress visibility), station-revenue with trip_count
routers/audit.py	GET /audit with date/type filters, DELETE /audit (all), DELETE /audit/{id} (single entry)

8. File Directory

Backend

File	One-line description
patna_metro_schema.sql	Complete DB: 13 tables, 4 stored procedures, 3 views, composite indexes, seed data
backend/main.py	Entry point — registers 11 routers, CORS middleware, lifespan init
backend/.env	DB credentials — DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME
backend/requirements.txt	fastapi, uvicorn, mysql-connector-python, python-dotenv, pydantic (v1)
backend/utils/database.py	Pool (autocommit=True) for reads + dedicated connection (autocommit=False) for all stored proc calls
backend/models/schemas.py	All Pydantic models — 18 request/response schemas
backend/routers/*.py	11 router files — one per domain (stations, lines, fares, users, cards, trips, machines, tickets, recharges, audit, analytics)

Frontend

File	One-line description
laptop1/src/api.js	Shared API client — all 3 apps use this; change BASE_URL once for multi-laptop WiFi
laptop1/src/App.jsx	Admin: Revenue charts, Trips, Tickets, Manage DB (stations/fares/users/cards/machines), Audit Log
laptop2/src/App.jsx	Vending: 3-step wizard, station grid, fare preview, ticket print — ?machine=N URL param
laptop4/src/App.jsx	Travel Card: PIN login, tap-in/tap-out with station selector, recharge presets, trip history

9. Setup & Run Instructions

9.1 Database

14. Open MySQL Workbench
15. File > Open SQL Script > select patna_metro_schema.sql
16. Click the lightning bolt (Execute) button
17. Verify: should see '10 stations loaded' style output in the log panel

9.2 Backend

Requirements: Python 3.11 (not 3.12 or 3.13 — pydantic-core has no wheel for those on Windows).

```
cd patna_metro_v2/backend
py -3.11 -m venv dbmsenv
dbmsenv\Scripts\activate
pip install -r requirements.txt
```

Edit .env and set DB_PASSWORD to your MySQL root password. Then:

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Verify at <http://localhost:8000/docs> — should show all routes and '10 stations loaded' in the terminal.

9.3 Frontend — Each Laptop

```
cd patna_metro_v2/frontend/laptop1
npx create-react-app .
npm install recharts # laptop 1 only
```

Copy the provided src/App.jsx and src/api.js into the src/ folder, replacing the defaults. Then:

```
npm start
```

For multi-laptop WiFi: find the backend laptop's IP (ipconfig in CMD), then in each api.js change:

```
export const BASE_URL = "http://<backend-laptop-IP>:8000";
```

Also run this in CMD on the backend laptop to allow port 8000 through Windows Firewall:

```
netsh advfirewall firewall add rule name="FastAPI" dir=in action=allow protocol=TCP
localport=8000
```

9.4 Vending Machine URL

Two kiosks can run from the same App.jsx. Open the browser to:

- <http://localhost:3000?machine=1> — binds to machine record with ID 1
- <http://localhost:3000?machine=2> — binds to machine record with ID 2

10. Seed Data

User	PIN	Card Number ends	Balance
Rahul Kumar	1234	3456	Rs. 250
Priya Singh	2345	4567	Rs. 150
Amit Sharma	3456	5678	Rs. 50

Admin password: admin123 | Fares: Rs. 15 to Rs. 30 | Minimum balance to travel: Rs. 10 | Ticket validity: 4 hours from issue | Vending machines: 5 (IDs 1 to 5)

11. Key Issues Fixed in v2.0

Issue	Fix Applied
Stored proc OUT params returning NULL	Dedicated autocommit=False connection for all call_proc calls — session variables guaranteed on same connection
Ticket not showing after booking	Step 3 render made independent of isOnline; polling paused at step 3; station names enriched from local state as fallback
Admin endless loader / blank screen	Separated initial load (shows spinner) from background polls (silent); error banner with RETRY added
Revenue charts empty with date filter	Switched daily-revenue grouping from exit_time to entry_time so in-progress trips appear immediately
Date presets off by one day	Fixed today() and daysAgo() to use local timezone (getFullYear/getMonth/getDate) instead of UTC toISOString()
station-revenue LEFT JOIN bug	Moved date filter from WHERE clause into JOIN ON condition so all stations appear even with zero trips
DB password error (1045 Access Denied)	Set DB_PASSWORD in backend/.env file matching your MySQL root password
Backend analytics 200 OK but no data	Default date range changed from daysAgo(29) to All Time (empty string) so data always displays on login

12. One-Line Summary

The application is a messenger. The database is the brain — **normalization prevents redundancy, constraints prevent invalid states, stored procedures with row locks prevent race conditions, and the WAL prevents data loss.**

End of Working Document | Patna Metro Management System | v2.0 | 2026